

Übungsblatt 5

Programmierung von Web-Systemen in Java

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Lernziele

- Die Komponenten des **Java Web Stack** (Servlet-Container, Build-Tools, Test-Frameworks) einordnen
- **Servlets**, **JSP** und **JavaBeans** als Grundbausteine kennen und einsetzen
- **Spring Boot** für REST-APIs und **Spring Data / JPA** für Persistenz anwenden
- Den **Spring Request-Flow** (DispatcherServlet → Controller → Service → Repository) verstehen
- **Dependency Injection** und die **Test-Pyramide** (Unit / Integration / E2E) anwenden

Modul A — Java Web Stack

Übung 5.A.1: Java Web Stack Begriffe

Ordnen Sie die folgenden Technologien der richtigen Kategorie im Java Web Stack zu und geben Sie eine kurze Beschreibung ihrer Aufgabe:

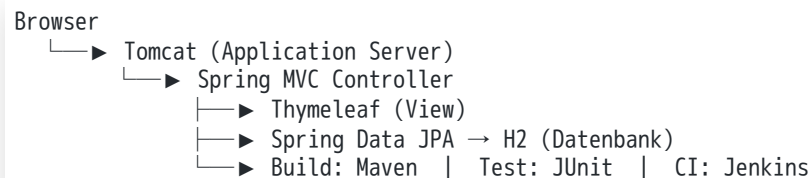
Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	?	?
Tomcat	?	?
Maven	?	?
JUnit	?	?
Jenkins	?	?
H2	?	?
Spring Framework	?	?

Mögliche Kategorien: View Template Engine, Application Server, Build Management Tool, Test Framework, CI/CD Tool, Datenbank, Web Framework

Übung 5.A.1: Java Web Stack Begriffe — Lösung

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Templates werden im Browser ohne Server darstellbar (Natural Templates)
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven	Build Management Tool	Verwaltet Abhängigkeiten, steuert den Build-Lifecycle per <code>pom.xml</code>
JUnit	Test Framework	Standard-Framework für Unit- und Integrationstests in Java
Jenkins	CI/CD Tool	Automatisierungsserver für kontinuierliche Integration und Deployment
H2	Datenbank	Leichtgewichtige In-Memory-Datenbank für Entwicklung und Tests
Spring Framework	Web Framework	Umfassendes Java-Framework: IoC-Container, MVC, Security, Data u.v.m.

Zusammenhang im Stack:



Übung 5.A.2: Maven Grundlagen

1. Was ist ein Maven-Artefakt? Wie ist eine Maven-Koordinate aufgebaut? Geben Sie ein Beispiel.
2. Was bedeutet semantische Versionierung (Semantic Versioning)? Erklären Sie MAJOR.MINOR.PATCH anhand eines Beispiels.
3. Nennen Sie mindestens vier Phasen des Maven Default Lifecycle in der richtigen Reihenfolge.

Übung 5.A.2: Maven Grundlagen — Lösung

1. Maven-Artefakt und Koordinate

Ein Maven-Artefakt ist das Ergebnis eines Builds (z.B. `.jar`, `.war`). Jedes Artefakt wird durch eine eindeutige Koordinate identifiziert:

```
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
<version>3.2.4</version>
```

Kurznotation: `groupId:artifactId:version`.

2. Semantische Versionierung (SemVer)

Teil	Bedeutung	Beispiel
MAJOR	Inkompatible API-Änderung	2.0.0 → 3.0.0
MINOR	Neue Funktion, abwärtskompatibel	3.1.0 → 3.2.0
PATCH	Bugfix, abwärtskompatibel	3.2.3 → 3.2.4

3. Maven Default Lifecycle

validate → compile → test → package → verify → install → deploy



Jede Phase schließt alle vorigen ein.

Modul B — Servlets

Übung 5.B.1: Servlet Lifecycle

Ordnen Sie die folgenden Code-Schnipsel dem richtigen Schritt im Servlet-Lebenszyklus zu. Beschreiben Sie außerdem, was diesen Schritt auslöst.

```
// Schnipsel A
protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<h1>Hello " + req.getParameter("name") + "</h1>");
}

// Schnipsel B
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    // einmalige Initialisierung
}

// Schnipsel C
public void destroy() {
    // Ressourcen freigeben
}

// Schnipsel D
@WebServlet(urlPatterns = {"/hello"})
```

Beantworten Sie außerdem: Welche Methode ruft der Container auf, wenn eine HTTP-POST-Anfrage eintrifft?



Übung 5.B.1: Servlet Lifecycle — Lösung

Schnipsel	Lifecycle-Schritt	Auslöser
D	Deployment / Registrierung	Container liest <code>@WebServlet</code> oder <code>web.xml</code>
B	<code>init()</code>	Einmalig nach Erzeugung des Servlet-Objekts
A	<code>service()</code> → <code>doGet()</code>	Bei jeder eingehenden HTTP-GET-Anfrage
C	<code>destroy()</code>	Beim Herunterfahren oder Undeploy

Bei einer HTTP-POST-Anfrage delegiert `service()` an `doPost()`.

Übung 5.B.2: Session Tracking Vergleich

Vergleichen Sie die fünf gängigen Session-Tracking-Methoden. Füllen Sie die Tabelle aus:

Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung (URL Rewriting)	?	?	?
Hidden Fields	?	?	?
Cookies	?	?	?



Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
HTTP Authorization Header	?	?	?
Session Tracking API (HttpSession)	?	?	?

Welche Methode nutzt Spring Boot's `HttpSession` standardmäßig?

Übung 5.B.2: Session Tracking Vergleich — Lösung

Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Gering	Fallback wenn Cookies deaktiviert sind
Hidden Fields	Ja	Mittel	Mehrstufige Formulare
Cookies	Nein	Hoch	Standard-Session-Tracking
HTTP Authorization Header	Ja	Gering	REST-APIs, JWT
Session Tracking API	Nein / Ja im Fallback	Mittel	Java EE / Jakarta EE Webapplikationen

Spring Boot verwendet standardmäßig einen **Cookie mit dem Namen JSESSIONID**.

Übung 5.B.3: JSP und Java Beans

Ein Formular auf `form.html` sendet eine POST-Anfrage an `/convert` mit dem Parameter `amount`. Das Servlet soll den Betrag von EUR in USD umrechnen (Kurs: 1 EUR = 1.10 USD) und das Ergebnis über eine JSP (`result.jsp`) ausgeben.

1. Skizzieren Sie den Request-Verarbeitungsfluss: Formular → Servlet → JSP.
2. Wie übergibt das Servlet das Ergebnis an die JSP, ohne es in der URL zu kodieren?
3. Welche Methode des Servlets (`doGet` oder `doPost`) wird aufgerufen, und warum?
4. Was ist ein **Java Bean** im JSP-Kontext, und welche Voraussetzungen muss eine Klasse erfüllen?

Übung 5.B.3: JSP und Java Beans — Lösung

1. Verarbeitungsfluss:

```
Browser: POST /convert (amount=100)
↓
ConvertServlet.doPost(request, response)
├── amount = request.getParameter("amount")
├── result = Double.parseDouble(amount) * 1.10
├── request.setAttribute("result", result)
└── forward("/WEB-INF/result.jsp")
↓
result.jsp: rendert HTML mit ${result}
```

2. Ergebnis an JSP übergeben:

```
request.setAttribute("result", amount * 1.10);
RequestDispatcher rd = request.getRequestDispatcher("/WEB-INF/result.jsp");
rd.forward(request, response);
```

3. Methode: `doPost()` — das Formular sendet `method="POST"`.

4. **Java Bean:** Eine normale Java-Klasse mit parameterlosem Konstruktor sowie öffentlichen Getter/Setter-Methoden; optional `Serializable`.

Übung 5.B.4: CurrencyServlet — eigene REST-API

In Übungsblatt 4 haben Sie einen Währungsrechner gegen die `exchangerate.host`-API gebaut. Implementieren Sie nun eine eigene Servlet-basierte API, die das gleiche Format liefert:

- GET /currency → JSON mit Wechselkursen von **EUR** in Fremdwährungen (USD, CHF)
- GET /currency?base=USD → Wechselkurse von **USD** in Fremdwährungen
- Wechselkurse dürfen im Quelltext fest verdrahtet werden

Implementieren Sie die `doGet()`-Methode des `CurrencyServlet` mit `@WebServlet`-Annotation. Welche Bibliothek verwenden Sie für die JSON-Serialisierung, und warum sollten Sie nicht von `String.format` benutzen?

Übung 5.B.4: CurrencyServlet — Lösung

```
@WebServlet(urlPatterns = {"/currency"})
public class CurrencyServlet extends HttpServlet {

    private static final Map<String, Map<String, Double>> RATES = Map.of(
        "EUR", Map.of("USD", 1.10, "CHF", 0.96),
        "USD", Map.of("EUR", 0.91, "CHF", 0.87)
    );

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        String base = Optional.ofNullable(req.getParameter("base")).orElse("EUR");
        Map<String, Double> rates = RATES.get(base);

        if (rates == null) {
            res.sendError(HttpServletResponse.SC_BAD_REQUEST,
                "Unsupported base currency: " + base);
            return;
        }

        Map<String, Object> payload = Map.of(
            "base", base,
            "date", LocalDate.now().toString(),
            "rates", rates
        );

        res.setContentType("application/json");
        res.setCharacterEncoding("UTF-8");
```

JSON-Bibliothek: Üblich ist **Jackson** (`com.fasterxml.jackson.databind.ObjectMapper`) oder **Gson**. Manuelles Zusammenbauen mit `String.format` ist fehleranfällig (Escaping von Anführungszeichen, Sonderzeichen, Unicode), produziert ungültiges JSON bei Edge Cases und ist schwer zu warten.

Maven-Abhängigkeit (Jackson):



```
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId>
```


Übung 5.B.5: JSP-Frontend mit FinanceBean

Erweitern Sie das CurrencyServlet aus Übung 5.B.4 um eine `doPost()`-Methode, die Formular-Daten von einer HTML-Seite entgegennimmt und das Ergebnis über eine JSP rendert.

Gegeben:

```
<!-- index.html -->
<form action="currency" method="POST">
  <input name="value" type="number" value="100"/>
  <select name="to">
    <option value="USD">USD</option>
    <option value="EUR">EUR</option>
  </select>
  <button type="submit">Umrechnen</button>
</form>

<!-- solution.jsp -->
<%@ page import="de.unipassau.currency.FinanceBean" %>
<% FinanceBean result = (FinanceBean) request.getAttribute("result"); %>
<h1>Ergebnis: <%= result.getAmount() %> <%= result.getCurrency() %></h1>
```

1. Implementieren Sie die FinanceBean-Klasse.
2. Implementieren Sie `doPost()`: liest `value` und `to`, rechnet um, befüllt die Bean, hängt sie als Request-Attribut `result` an und leitet auf `solution.jsp` weiter.

Übung 5.B.5: JSP + FinanceBean — Lösung

FinanceBean (parameterloser Konstruktor + Getter/Setter):

```
package de.unipassau.currency;

public class FinanceBean implements java.io.Serializable {
    private double amount;
    private String currency;

    public FinanceBean() { }

    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }

    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }
}
```

doPost im CurrencyServlet:

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    double value = Double.parseDouble(req.getParameter("value"));
    String to = req.getParameter("to");

    double rate = RATES.get("EUR").getOrDefault(to, 1.0);
    if ("EUR".equals(to)) rate = 1.0;

    FinanceBean result = new FinanceBean();
    result.setAmount(value * rate);
    result.setCurrency(to);

    req.setAttribute("result", result);
    req.getRequestDispatcher("/solution.jsp").forward(req, res);
}
```



Forward statt Redirect: `RequestDispatcher.forward()` behält Request-Attribute (die JSP findet result). Ein `sendRedirect` würde einen neuen Request triggern und die Bean wäre weg.

Modul C — Spring Boot & REST

Übung 5.C.1: Spring Boot REST-Controller

Implementieren Sie einen Spring Boot REST-Controller für ein Studierenden-Verwaltungssystem:

- GET /students — alle Studierenden zurückgeben
- GET /students/{id} — Studierendem mit ID; 404 wenn nicht vorhanden
- POST /students — neuen Studierenden anlegen; 201 mit Location-Header
- DELETE /students/{id} — Studierenden löschen; 204 bei Erfolg

Verwenden Sie eine `StudentRepository`-Schnittstelle. Welche Spring-Annotationen benötigen Sie?

Übung 5.C.1: Spring Boot REST-Controller — Lösung

```
@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentRepository repo;

    public StudentController(StudentRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<Student> getAll() {
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable Long id) {
        return repo.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Student> create(@RequestBody @Valid Student student,
                                           UriComponentsBuilder ucb) {
        Student saved = repo.save(student);
        URI location = ucb.path("/students/{id}")
            .buildAndExpand(saved.getId()).toUri();
    }
}
```

Wichtige Annotationen: @RestController, @RequestMapping, @PathVariable, @RequestBody, @Valid.

Übung 5.C.2: Dependency Injection und Testbarkeit

Vergleichen Sie folgende zwei Implementierungen:



```
// Version A
@Service
```

```
public class OrderService {  
    private final EmailService email = new EmailService();  
    public void placeOrder(Order o) { email.send(o); }  
}  
  
// Version B  
@Service  
public class OrderService {  
    private final EmailService email;  
    public OrderService(EmailService email) { this.email = email; }  
    public void placeOrder(Order o) { email.send(o); }  
}
```

1. Welche Version ist testbarer? Warum?
2. Was macht Spring automatisch bei Version B?
3. Schreiben Sie einen Unit-Test für Version B, der EmailService mockt.

Übung 5.C.2: Dependency Injection und Testbarkeit — Lösung

1. **Version B ist testbarer.** Der EmailService wird von außen injiziert → im Test kann ein Mock übergeben werden.
2. Spring injiziert automatisch die passende Bean per Constructor Injection.

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    EmailService emailService;

    @InjectMocks
    OrderService orderService;

    @Test
    void placeOrder_sendsEmail() {
        Order order = new Order(42L, "Laptop");
        orderService.placeOrder(order);
        verify(emailService, times(1)).send(order);
    }
}
```

Übung 5.C.3: Test-Pyramide

Beschreiben Sie die drei Ebenen der Test-Pyramide für eine Spring Boot Anwendung:

1. Was testet jede Ebene? Nennen Sie ein konkretes Beispiel.
2. Wie verhält sich die empfohlene Anzahl der Tests von der untersten zur obersten Ebene?
3. Welche Spring-Test-Annotationen oder -Bibliotheken sind typisch?

Übung 5.C.3: Test-Pyramide — Lösung

1. **Unit Tests:** einzelne Klasse oder Methode isoliert, Abhängigkeiten gemockt.
2. **Integrationstests:** Zusammenspiel mehrerer Komponenten, z.B. Controller + JSON.
3. **End-to-End-Tests:** gesamter Stack inkl. HTTP und echter Datenbank.

Faustregel: unten viele Tests, oben wenige.

Übung 5.C.4: MVC, MVP und MVVM im Vergleich

1. Füllen Sie die Tabelle aus.
2. Wann wählt man Spring MVC (Thymeleaf) und wann React + Spring REST?
3. Warum ist MVP besonders testfreundlich?

Übung 5.C.4: MVC, MVP und MVVM im Vergleich — Lösung

Aspekt	MVC	MVP	MVVM
Wer kommuniziert mit dem Model?	Controller und View direkt	Nur der Presenter	Nur das ViewModel
Reaktion auf Änderungen	Direkt / Observer	Presenter aktualisiert View	Data Binding
Testbarkeit	gut	sehr gut	gut
Typische Plattform	Spring MVC	Android	Angular, React, Vue

Entscheidung:

- **Spring MVC + Thymeleaf** für serverseitiges Rendering und SEO.
- **React + Spring REST** für komplexe, interaktive Client-Anwendungen.

MVP ist testfreundlich, weil die View nur als Interface vorkommt und der Presenter ohne UI-Klassen getestet werden kann.

Übung 5.C.5: Spring CommandLineRunner

Bei der Entwicklung einer Spring Boot-Anwendung sollen beim Start automatisch Testdaten in die Datenbank geladen werden.



1. Was ist ein `CommandLineRunner` in Spring Boot, und wann wird er ausgeführt?
2. Implementieren Sie einen `CommandLineRunner`, der beim Start 3 Produkte in ein `ProductRepository` (JPA) lädt.
3. Was ist der Unterschied zwischen `CommandLineRunner` und `ApplicationRunner`?

Übung 5.C.5: Spring CommandLineRunner — Lösung

1. CommandLineRunner wird nach dem vollständigen Start des ApplicationContext ausgeführt.
2. Implementierung:

```
@Component
public class DataInitializer implements CommandLineRunner {

    private final ProductRepository productRepository;

    public DataInitializer(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Override
    public void run(String... args) {
        productRepository.saveAll(List.of(
            new Product("Laptop", 999.99),
            new Product("Mouse", 29.99),
            new Product("Keyboard", 79.99)
        ));
    }
}
```

3. CommandLineRunner bekommt ein String-Array, ApplicationRunner ein strukturiertes ApplicationArguments-Objekt.

Modul D — Persistenz mit JPA

Übung 5.D.1: Entity-Mapping und Repository

Modellieren Sie eine JPA-Entity Course mit folgenden Attributen:

- id (automatisch generiert)
- title (nicht null, max. 200 Zeichen)
- credits (Ganzzahl, 1–10)
- instructor (String)

Erstellen Sie dazu ein `CourseRepository` mit einer Methode, die alle Kurse eines bestimmten Dozenten findet.

Welche JPA-Annotationen und welche Spring Data-Schnittstelle verwenden Sie?

Übung 5.D.1: Entity-Mapping und Repository — Lösung

```
@Entity
@Table(name = "courses")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(nullable = false)
    @Min(1) @Max(10)
    private int credits;

    private String instructor;
}

public interface CourseRepository extends JpaRepository<Course, Long> {
    List<Course> findByInstructor(String instructor);
}
```

Übung 5.D.2: Kursanmeldung mit Voraussetzungsprüfung

Implementieren Sie `EnrollmentService.enroll(studentId, courseId)` mit folgenden Regeln:

1. Student muss existieren
2. Kurs muss existieren
3. Student darf noch nicht eingeschrieben sein
4. Student muss alle Voraussetzungskurse abgeschlossen haben

Welchen HTTP-Status-Code soll der Controller für jede Exception zurückgeben?

Übung 5.D.2: Kursanmeldung mit Voraussetzungsprüfung — Lösung

```
@Service
@Transactional
public class EnrollmentService {

    public Enrollment enroll(Long studentId, Long courseId) {
        Student student = studentRepo.findById(studentId)
            .orElseThrow(() -> new StudentNotFoundException(studentId));
        Course course = courseRepo.findById(courseId)
            .orElseThrow(() -> new CourseNotFoundException(courseId));

        if (enrollmentRepo.existsByStudentAndCourse(student, course))
            throw new AlreadyEnrolledException(studentId, courseId);

        Set<Course> completed = enrollmentRepo.completedCourses(student);
        if (!completed.containsAll(course.getPrerequisites()))
            throw new MissingPrerequisiteException(courseId);

        return enrollmentRepo.save(new Enrollment(student, course));
    }
}
```

Exception	HTTP-Status
StudentNotFoundException	404 Not Found
CourseNotFoundException	404 Not Found
AlreadyEnrolledException	409 Conflict
MissingPrerequisiteException	400 Bad Request

Übung 5.D.3: JPA Vererbungsstrategien

Sie modellieren eine Klassenhierarchie: Person, Student und Lehrkraft.

Skizzieren Sie, wie die drei JPA-Vererbungsstrategien diese Hierarchie in Datenbanktabellen abbilden würden:

1. SINGLE_TABLE
2. JOINED
3. TABLE_PER_CLASS

Welche Strategie würden Sie für dieses Szenario wählen und warum?

Übung 5.D.3: JPA Vererbungsstrategien — Lösung

SINGLE_TABLE: Eine Tabelle mit allen Spalten und einem Diskriminator. Schnell, aber viele NULL-Werte.

JOINED: Eine Basistabelle plus Unterklassen-Tabellen. Normalisiert, aber JOINS.

TABLE_PER_CLASS: Eine Tabelle pro konkreter Klasse. Redundanz, polymorphe Queries schwierig.

Empfehlung: JOINED ist für dieses Szenario die beste Wahl, weil die Hierarchie flach ist und die Unterklassen eigene Felder haben.

Materialien zum Übungsblatt

Zur Übung mit Servlets, JSP und Spring Boot stehen vollständige Code-Templates bereit:

Servlets / JSP

- [servlets-template/](#) — minimales Servlet-Template (CurrencyServlet.java, FinanceBean.java, index.html, api.html, TestCurrencyServlet.java) zum selbst Ausfüllen
- [servlets-filled/](#) — funktionierende Referenzlösung

Spring Boot REST

- [spring-boot-rest/](#) — vollständige Studenten-Service-Anwendung mit StudentController, StudentService, Student, Course, DTOs, Initialisierung und Tests

*Alle Materialien finden Sie auch zentral auf der **Kursseite** unter Materials → Download.*